

Phase 3: Model 2 - Association Rule Mining & Classification Based on Associations

Why ARM-CBA for Railroad Accident Classification?

Association Rule Mining (ARM) was originally developed for market basket analysis - discovering that customers who buy bread and butter also tend to buy milk (Agrawal, Imielinski, & Swami, 1993). The technique finds groups of items that frequently appear together in transaction data. It has since been applied across domains wherever categorical co-occurrence patterns matter, from occupational safety in construction (Liao & Perng, 2008) and steel plants (Verma et al., 2014) to traffic crash analysis in China (Xu, Bao, Wang, & Liu, 2018).

The idea to apply ARM to railroad accident classification came from two sources: DSCI 521 Week 4 lecture notes on Association Rule Mining, and reading Xu et al. (2018), who used ARM to analyze factors contributing to severe traffic crashes in China. Their approach treated each crash as a transaction and each contributing factor as an item, then mined association rules to discover which combinations of factors tend to appear together in serious accidents.

Our project adapts this same concept to FRA railroad accident data. Each accident is a transaction. Each feature value (e.g., Track Type = Main, Weather = Rain) is an item. And the cause category (Environmental, Human Factor, Mechanical, Signal, Track) is the item we want to predict. ARM discovers which combinations of environmental and operational conditions co-occur with specific cause categories. CBA then takes those discovered rules and organizes them into a classifier.

What makes this approach novel for our project is that ARM and CBA are not standard classification algorithms like Logistic Regression or Random Forest. They come from a completely different tradition - data mining and pattern discovery rather than statistical modeling. Applying a technique built for retail shopping baskets to railroad safety analysis demonstrates that the underlying principle (finding meaningful co-occurrence patterns in categorical data) transfers across domains.

Two Distinct Steps

This notebook has two distinct phases that should not be conflated:

1. **Association Rule Mining (ARM):** The Apriori algorithm scans the training data and discovers which feature combinations frequently co-occur with each cause category. ARM is a data generation step. Its output is a set of rules (with support,

confidence, and lift metrics) that is used as a feature in the down-stream CBA analysis. These rules are saved to CSV as a standalone, reusable artifact.

2. **Classification Based on Associations (CBA):** The CBA classifier takes ARM's output, ranks the rules, and uses first-match logic to classify new accidents. CBA is a modeling step that consumes ARM's rules. It is a separate algorithm described by Liu, Hsu, & Ma (1998).

ARM finds the patterns (creates the features). CBA uses them to make predictions. They are related but independent steps.

References

- Agrawal, R., Imielinski, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *Proceedings of the ACM SIGMOD*, 207-216.
 - DSCI 521 Lecture Notes, Week 4: Association Rule Mining.
 - Liu, B., Hsu, W., & Ma, Y. (1998). Integrating Classification and Association Rule Mining. *KDD-98*, 80-86.
 - Xu, C., Bao, J., Wang, C., & Liu, P. (2018). Association rule analysis of factors contributing to extraordinarily severe traffic crashes in China. *Journal of Safety Research*, 67, 65-75.
-

Setup

Import required libraries and set random seed for reproducibility. All random operations throughout this notebook use `RANDOM_SEED = 521` to ensure consistent results across runs.

All plots use the `tableau-colorblind10` colour cycle following CTAO best practices for colour-blind-friendly publications (CTAO, 2020). Cause categories are assigned fixed colours and hatch patterns so they can be identified through multiple visual channels.

```
In [2]: import pandas as pd
import numpy as np
import json
import joblib
import warnings

df_merged = pd.read_csv("../data/processed/df_merged_features_complete.csv.z
df_encoded = pd.read_csv("../data/processed/df_encoded_features_complete.csv

print()
print("Verify columns in df_merged")
pprint(df_merged.columns.tolist())
```

```
C:\Users\David\AppData\Local\Temp\ipykernel_19636\3595987982.py:4: DtypeWarning: Columns (26,28,48,68,71) have mixed types. Specify dtype option on import or set low_memory=False.  
    df_merged = pd.read_csv("../data/processed/df_merged_features_complete.csv.zip")
```

```
Verify columns in df_merged
['Reporting Railroad Code',
 'Reporting Railroad Name',
 'Year',
 'Accident Number',
 'PDF Link',
 'Accident Year',
 'Accident Month',
 'Other Railroad Code',
 'Other Railroad Name',
 'Other Accident Number',
 'Other Accident Year',
 'Other Accident Month',
 'Maintenance Railroad Code',
 'Maintenance Railroad Name',
 'Maintenance Accident Number',
 'Maintenance Accident Year',
 'Maintenance Accident Month',
 'Grade Crossing ID',
 'Day',
 'Date',
 'Time',
 'Accident Type Code',
 'Accident Type',
 'Hazmat Cars',
 'Hazmat Cars Damaged',
 'Hazmat Released Cars',
 'Persons Evacuated',
 'Subdivision',
 'Division Code',
 'Division',
 'Station',
 'Milepost',
 'State Code',
 'State Abbreviation',
 'State Name',
 'County Code',
 'County Name',
 'District',
 'Temperature',
 'Visibility Code',
 'Visibility',
 'Weather Condition Code',
 'Weather Condition',
 'Track Type Code',
 'Track Type',
 'Track Name',
 'Track Class',
 'Track Density',
 'Train Direction Code',
 'Train Direction',
 'Equipment Type Code',
 'Equipment Type',
 'Equipment Attended',
 'Train Number',
 'Train Speed',
```

'Recorded Estimated Speed',
'Maximum Speed',
'Gross Tonnage',
'Signalization Code',
'Signalization',
'Method of Operation Code',
'Method of Operation',
'Adjunct Code 1',
'Adjunct Name 1',
'Adjunct Code 2',
'Adjunct Name 2',
'Adjunct Code 3',
'Adjunct Name 3',
'Remote Control Locomotive Code',
'Remote Control Locomotive',
'First Car Initials',
'First Car Number',
'First Car Position',
'First Car Loaded',
'Causing Car Initials',
'Causing Car Number',
'Causing Car Position',
'Causing Car Loaded',
'Positive Alcohol Tests',
'Positive Drug Tests',
'Passengers Transported',
'Head End Locomotives',
'Mid Train Manual Locomotives',
'Mid Train Remote Locomotives',
'Rear End Manual Locomotives',
'Rear End Remote Locomotives',
'Derailed Head End Locomotives',
'Derailed Mid Train Manual Locomotives',
'Derailed Mid Train Remote Locomotives',
'Derailed Rear End Manual Locomotives',
'Derailed Rear End Remote Locomotives',
'Loaded Freight Cars',
'Loaded Passenger Cars',
'Empty Freight Cars',
'Empty Passenger Cars',
'Cabooses',
'Derailed Loaded Freight Cars',
'Derailed Loaded Passenger Cars',
'Derailed Empty Freight Cars',
'Derailed Empty Passenger Cars',
'Derailed Cabooses',
'Equipment Damage Cost',
'Track Damage Cost',
'Total Damage Cost',
'Primary Accident Cause Code',
'Primary Accident Cause',
'Contributing Accident Cause Code',
'Contributing Accident Cause',
'Accident Cause Code',
'Accident Cause',
'Engineers On Duty',

'Firemen On Duty',
'Conductors On Duty',
'Brakemen On Duty',
'Hours Engineers On Duty',
'Minutes Engineers On Duty',
'Hours Conductors On Duty',
'Minutes Conductors On Duty',
'Railroad Employees Killed',
'Railroad Employees Injured',
'Passengers Killed',
'Passengers Injured',
'Others Killed',
'Others Injured',
'Reporting Railroad Fatalities for 55a',
'Reporting Railroad Injuries for 55a',
'Total Persons Killed',
'Total Persons Injured',
'Total Killed Form 54',
'Total Injured Form 54',
'Special Study 1',
'Special Study 2',
'Latitude',
'Longitude',
'Narrative',
'Joint Track Type',
'Joint Track Class',
'Class Code',
'Class',
'Joint CD',
'Incident Key',
'Report Key',
'Reporting Railroad Class',
'Reporting Railroad SMT Grouping',
'Reporting Parent Railroad Code',
'Reporting Parent Railroad Name',
'Reporting Railroad Holding Company',
'Reporting Railroad Individual Class',
'Reporting Railroad Passenger',
'Reporting Railroad Commuter',
'Reporting Railroad Switching Terminal',
'Reporting Railroad Tourist',
'Reporting Railroad Freight',
'Reporting Railroad Short Line',
'Location',
'State Code FIPS',
'County Code FIPS',
'FIPS',
'Description',
'Population_2020',
'RUCC_2023',
'Cause_Category',
'total_cars',
'empty_loaded_ratio',
'pct_empty',
'tonnage_per_car',
'Equipment_Type_Mapped',

```
'Day_of_Week',  
'Hour',  
'Time_of_Day',  
'Time_parsed',  
'Region',  
'Cause_Name']
```

```
C:\Users\David\AppData\Local\Temp\ipykernel_19636\3595987982.py:5: DtypeWarning: Columns (26,28,48,68,71) have mixed types. Specify dtype option on import or set low_memory=False.
```

```
df_encoded = pd.read_csv("../data/processed/df_encoded_features_complete.csv.zip")
```

```
In [3]: print("Verify columns in df_encoded")  
pprint(df_encoded.columns.tolist())
```

```
Verify columns in df_encoded
['Reporting Railroad Code',
 'Reporting Railroad Name',
 'Year',
 'Accident Number',
 'PDF Link',
 'Accident Year',
 'Accident Month',
 'Other Railroad Code',
 'Other Railroad Name',
 'Other Accident Number',
 'Other Accident Year',
 'Other Accident Month',
 'Maintenance Railroad Code',
 'Maintenance Railroad Name',
 'Maintenance Accident Number',
 'Maintenance Accident Year',
 'Maintenance Accident Month',
 'Grade Crossing ID',
 'Day',
 'Date',
 'Time',
 'Accident Type Code',
 'Accident Type',
 'Hazmat Cars',
 'Hazmat Cars Damaged',
 'Hazmat Released Cars',
 'Persons Evacuated',
 'Subdivision',
 'Division Code',
 'Division',
 'Station',
 'Milepost',
 'State Code',
 'State Abbreviation',
 'State Name',
 'County Code',
 'County Name',
 'District',
 'Temperature',
 'Visibility Code',
 'Visibility',
 'Weather Condition Code',
 'Weather Condition',
 'Track Type Code',
 'Track Type',
 'Track Name',
 'Track Class',
 'Track Density',
 'Train Direction Code',
 'Train Direction',
 'Equipment Type Code',
 'Equipment Type',
 'Equipment Attended',
 'Train Number',
 'Train Speed',
```


'Recorded Estimated Speed',
'Maximum Speed',
'Gross Tonnage',
'Signalization Code',
'Signalization',
'Method of Operation Code',
'Method of Operation',
'Adjunct Code 1',
'Adjunct Name 1',
'Adjunct Code 2',
'Adjunct Name 2',
'Adjunct Code 3',
'Adjunct Name 3',
'Remote Control Locomotive Code',
'Remote Control Locomotive',
'First Car Initials',
'First Car Number',
'First Car Position',
'First Car Loaded',
'Causing Car Initials',
'Causing Car Number',
'Causing Car Position',
'Causing Car Loaded',
'Positive Alcohol Tests',
'Positive Drug Tests',
'Passengers Transported',
'Head End Locomotives',
'Mid Train Manual Locomotives',
'Mid Train Remote Locomotives',
'Rear End Manual Locomotives',
'Rear End Remote Locomotives',
'Derailed Head End Locomotives',
'Derailed Mid Train Manual Locomotives',
'Derailed Mid Train Remote Locomotives',
'Derailed Rear End Manual Locomotives',
'Derailed Rear End Remote Locomotives',
'Loaded Freight Cars',
'Loaded Passenger Cars',
'Empty Freight Cars',
'Empty Passenger Cars',
'Cabooses',
'Derailed Loaded Freight Cars',
'Derailed Loaded Passenger Cars',
'Derailed Empty Freight Cars',
'Derailed Empty Passenger Cars',
'Derailed Cabooses',
'Equipment Damage Cost',
'Track Damage Cost',
'Total Damage Cost',
'Primary Accident Cause Code',
'Primary Accident Cause',
'Contributing Accident Cause Code',
'Contributing Accident Cause',
'Accident Cause Code',
'Accident Cause',
'Engineers On Duty',

'Firemen On Duty',
'Conductors On Duty',
'Brakemen On Duty',
'Hours Engineers On Duty',
'Minutes Engineers On Duty',
'Hours Conductors On Duty',
'Minutes Conductors On Duty',
'Railroad Employees Killed',
'Railroad Employees Injured',
'Passengers Killed',
'Passengers Injured',
'Others Killed',
'Others Injured',
'Reporting Railroad Fatalities for 55a',
'Reporting Railroad Injuries for 55a',
'Total Persons Killed',
'Total Persons Injured',
'Total Killed Form 54',
'Total Injured Form 54',
'Special Study 1',
'Special Study 2',
'Latitude',
'Longitude',
'Narrative',
'Joint Track Type',
'Joint Track Class',
'Class Code',
'Class',
'Joint CD',
'Incident Key',
'Report Key',
'Reporting Railroad Class',
'Reporting Railroad SMT Grouping',
'Reporting Parent Railroad Code',
'Reporting Parent Railroad Name',
'Reporting Railroad Holding Company',
'Reporting Railroad Individual Class',
'Reporting Railroad Passenger',
'Reporting Railroad Commuter',
'Reporting Railroad Switching Terminal',
'Reporting Railroad Tourist',
'Reporting Railroad Freight',
'Reporting Railroad Short Line',
'Location',
'State Code FIPS',
'County Code FIPS',
'FIPS',
'Description',
'Population_2020',
'RUCC_2023',
'Cause_Category',
'total_cars',
'empty_loaded_ratio',
'pct_empty',
'tonnage_per_car',
'Equipment_Type_Mapped',

```

'Day_of_Week',
'Hour',
'Time_of_Day',
'Time_parsed',
'Region',
'Cause_Name',
'Weather Condition_encoded',
'Track Type_encoded',
'Visibility_encoded',
'Accident Type_encoded',
'Region_encoded',
'Equipment_Type_encoded',
'Day_of_Week_encoded',
'Time_of_Day_encoded',
'Signalization_encoded']

```

```

In [4]: # Select the shortlist categorical columns for one-hot encoding
cols_to_ohe = [
    'Weather Condition',
    'Track Type',
    'Visibility',
    'Accident Type',
    'Region',
    'RUCC_2023',
    'Equipment Type'
]

# Create df_ohe with one-hot encoded variables (used later for Association Rule)
# pd.get_dummies will automatically convert these categorical columns to 0/1
df_ohe = pd.get_dummies(
    df_merged[cols_to_ohe],
    columns=cols_to_ohe,
    dtype=int
)

import matplotlib.pyplot as plt
import matplotlib.style as style
import seaborn as sns

warnings.filterwarnings('ignore', category=FutureWarning)
style.use('tableau-colorblind10')

plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 10

RANDOM_SEED = 521
np.random.seed(RANDOM_SEED)

# Consistent visual identity for cause categories across all plots
CAUSE_STYLES = {
    'E': {'label': 'Environmental', 'hatch': '//'},
    'H': {'label': 'Human Factor', 'hatch': '..'},
    'M': {'label': 'Mechanical', 'hatch': 'xx'},
    'S': {'label': 'Signal', 'hatch': '\\\\'},
    'T': {'label': 'Track', 'hatch': '++'},
}

```

```

CAUSE_ORDER = ['E', 'H', 'M', 'S', 'T']

# -----
# ARM Threshold Configurations
# Defined once, used in both Part 1 (ARM) and Part 2 (CBA)
# to ensure experimental consistency between sections.
# -----
HIGH_SUPPORT = 0.05
HIGH_CONFIDENCE = 0.70
HIGH_LIFT = 2.0

LOW_SUPPORT = 0.005
LOW_CONFIDENCE = 0.40
LOW_LIFT = 1.5

MAX_RULE_LENGTH = 4

print(f"High thresholds: S={HIGH_SUPPORT}, C={HIGH_CONFIDENCE}, L={HIGH_LIFT}")
print(f"Low thresholds: S={LOW_SUPPORT}, C={LOW_CONFIDENCE}, L={LOW_LIFT}")
print(f"Max rule length: {MAX_RULE_LENGTH}")

```

Shape of df_ohe: (34562, 54)

```

['Weather Condition_Clear',
 'Weather Condition_Fog',
 'Weather Condition_Sleet',
 'Track Type_Industry',
 'Track Type_Siding',
 'Visibility_Dark',
 'Visibility_Day',
 'Accident Type_Broken train collision',
 'Accident Type_Explosion-detonation',
 'Accident Type_Head on collision',
 'Accident Type_Obstruction',
 'Accident Type_Other impacts',
 'Accident Type_Raking collision',
 'Accident Type_Side collision',
 'Region_Northeast',
 'Region_West',
 'RUCC_2023_2',
 'RUCC_2023_4',
 'RUCC_2023_6',
 'RUCC_2023_8',
 'Equipment Type_Commuter Train - Pulling',
 'Equipment Type_Cut of cars',
 'Equipment Type_EMU',
 'Equipment Type_Light loco(s)',
 'Equipment Type_Passenger Train - Pulling',
 'Equipment Type_Single Car',
 'Equipment Type_Work train']

```

Out [4]:

	Weather Condition_Clear	Weather Condition_Cloudy	Weather Condition_Fog	Weather Condition_Rain	Weather Condition_Sleet
0	1	0	0	0	0
1	1	0	0	0	0
2	1	0	0	0	0
3	1	0	0	0	0
4	1	0	0	0	0

5 rows × 54 columns

Load Training Data

Load the 80% training set from Phase 2. The 20% test set remains locked until final evaluation. Model selection and validation are handled entirely through cross-validation on this training set.

```
In [5]: df_train = pd.read_csv('../data/splits/train_80.csv', low_memory=False)

print(f"Training set: {len(df_train)} rows")
print(f"\nCause_Category distribution:")
print(df_train['Cause_Category'].value_counts(normalize=True).round(3))
```

Training set: 27640 rows

Cause_Category distribution:

Cause_Category

H 0.393

M 0.256

T 0.218

E 0.108

S 0.025

Name: proportion, dtype: float64

Feature Selection

Use the same 6 features as the LR and RF baselines for a direct apples-to-apples comparison across all three models.

ARM requires transactional input where each category level becomes its own column with True/False values indicating whether that item is present in the transaction. This is one-hot encoding (OHE), handled via `pd.get_dummies()`. For example, Track Type (4 levels) becomes four separate columns: Track Type_Main, Track Type_Yard, Track Type_Industry, Track Type_Siding. Each accident row has exactly one of these set to True and the rest False. This is distinct from binary encoding, which represents

categories as binary digits across fewer columns – ARM cannot use binary encoding because individual category presence/absence must be explicitly represented.

The target variable (Cause_Category) is also one-hot encoded and appended to the transaction table so ARM can discover rules where cause categories appear as consequents (e.g., {Track Type_Main, Weather_Rain} -> {Cause_M}).

Note: Accident Type is included in this baseline for comparison purposes. Phase 2 analysis identified it as a dominant confounding variable (Cramer's V = 0.39). ARM experiments in Phase 2 confirmed it dominates the rule space, producing zero rules when excluded at standard thresholds. A separate experiment will evaluate performance without it.

For a comparison of one-hot encoding, label encoding, and binary encoding tradeoffs, see Menear (2023): <https://medium.com/@kevin.menear/comparing-label-encoding-one-hot-encoding-and-binary-encoding-for-handling-categorical-variables-933544ccbd02>

```
In [5]: # Same 6 features as LR and RF baselines
feature_cols = [
    'Weather Condition',
    'Track Type',
    'Visibility',
    'Accident Type',
    'Region',
    'RUCC_2023'
]

target = 'Cause_Category'

X_train_raw = df_train[feature_cols].copy()
y_train = df_train[target].copy()

print(f"Features: {len(feature_cols)}")
print(f" {feature_cols}")
print(f"\nTarget classes: {sorted(y_train.unique())}")
print(f"Training samples: {len(X_train_raw)}")
```

Features: 6

['Weather Condition', 'Track Type', 'Visibility', 'Accident Type', 'Region', 'RUCC_2023']

Target classes: ['E', 'H', 'M', 'S', 'T']

Training samples: 27640

Part 1: Association Rule Mining (ARM)

Part 1 runs ARM as a standalone data generation step on the full training set. The purpose is to showcase ARM independently - examining the rules it discovers, analyzing which cause categories get coverage at different thresholds, and saving the rules as reusable CSV artifacts.

Part 2 (CBA) will re-run ARM internally inside each cross-validation fold. This is a design requirement: proper CV demands that rules be mined only from each fold's training split, never from validation data. The CBA classifier wraps ARM inside its `fit()` method so that each fold gets its own rule set mined from its own training data.

Both sections use the same threshold variables defined in Setup (`HIGH_SUPPORT` , `LOW_SUPPORT` , etc.) to ensure experimental consistency. Changing a threshold in one place changes it everywhere.

How Association Rule Mining Works

From Shopping Carts to Railroad Accidents

ARM works on **transaction data** - a collection of records where each record contains a set of items. In retail, a transaction is a shopping receipt and the items are products purchased. In our project, a transaction is a railroad accident and the items are the feature values that describe it: Track Type_Main, Weather Condition_Rain, Visibility_Dark, Region_South, RUCC_Metro_Adjacency_Metro, and the cause category Cause_M (Mechanical).

ARM's job is to discover which items frequently appear together. When a pattern like "Track Type_Main AND Weather Condition_Rain AND Cause_M" appears often enough, ARM reports it as a rule: "Main track + Rain -> Mechanical."

The Apriori Algorithm: Pruning the Search Space

The challenge is computational. With 5 categorical features that expand to 21 columns after one-hot encoding, plus 5 cause category columns, the total number of possible combinations is:

$$2^{26} - 1 = 67,108,863$$

Checking all 67 million subsets is not practical. The Apriori algorithm solves this with one key insight: **if a small pattern is rare, every larger pattern containing it is also rare.**

For example, if "Track Type_Siding" appears in only 4% of accidents and our minimum support threshold is 5%, then we immediately know that "Track Type_Siding AND Weather Condition_Snow" must have support of 4% or less. There is no reason to count it.

As the lecture notes describe using a lattice structure, Apriori works bottom-up:

1. Count each individual item's frequency. Keep only items meeting the minimum support threshold.
2. Combine surviving items into pairs. Count pairs. Keep only pairs meeting the threshold.
3. Combine surviving pairs into triples. Count triples. Keep only triples meeting the threshold.
4. Continue until no new combinations survive.

At each step, the algorithm only builds combinations from patterns that already passed. This prunes the exponential search space down to something manageable.

Three Metrics That Define a Rule

Every discovered rule has three quality metrics:

Support measures how common a pattern is. If "Track Type_Main AND Weather Condition_Rain AND Cause_M" appears in 1.2% of training rows, its support is 0.012. Low support means the pattern is rare - it might be a real but uncommon phenomenon, or it might be coincidence.

Confidence measures how reliable a rule is as a predictor. If the pattern "Main track + Rain" appears in 100 accidents, and 60 of those are Mechanical failures, the confidence for "Main track + Rain -> Mechanical" is 0.60. A confidence of 0.60 means the rule is correct 60% of the time when its conditions are met.

Lift measures improvement over random chance. If Mechanical failures account for 26% of all accidents, and our rule predicts Mechanical with 60% confidence, the lift is $0.60 / 0.26 = 2.3$. This means the rule is 2.3 times better than blindly guessing Mechanical. A lift below 1.0 means the rule is worse than guessing, which would make it useless.

The Class Imbalance Problem for ARM

ARM's support threshold creates a structural problem for rare classes. Support is calculated as a fraction of the total dataset. Signal accidents represent only 2.5% of our data, so the maximum possible support for any Signal rule is 0.025. At a minimum support threshold of 0.02, Signal patterns have almost no room to qualify - they would need to appear in nearly every Signal accident while also sharing the exact same feature combination.

This is a fundamental limitation of ARM on imbalanced data, and it is why the cba-tuning-experiments notebook introduces undersampling before rule mining. But for this baseline, we use the natural class distribution and observe what ARM can and cannot discover.

ARM Output: Rules as a Reusable Artifact

The Apriori algorithm produces a table of rules with their support, confidence, and lift values. This table is the output of the ARM step. It is saved to CSV in

`data/processed/arm_rules/` so it can be inspected independently, shared with teammates, and loaded by a non-cross-validated CBA classifier without re-mining.

ARM Experiment 1: High Threshold Rule Mining

Conservative parameters that prioritize rule reliability over coverage:

- **Support $\geq 5\%$:** Pattern must appear in at least 1 in 20 training rows
- **Confidence $\geq 70\%$:** Rule must be correct at least 70% of the time
- **Lift ≥ 2.0 :** Rule must be at least 2x better than random chance

These strict thresholds will produce fewer, higher-quality rules. The tradeoff is that many patterns, especially for minority classes like Signal (2.5%) and Environmental (11%), may not meet the support threshold and will never become rules.

```
In [6]: # ARM Step: Mine rules at high thresholds
from mlxtend.frequent_patterns import apriori, association_rules

# One-hot encode features into binary transaction format
X_ohe = pd.get_dummies(X_train_raw, columns=X_train_raw.columns.tolist(), dtype=bool)

# One-hot encode target and append
y_ohe = pd.get_dummies(y_train, prefix='Cause', dtype=bool)
cause_columns = y_ohe.columns.tolist()
df_arm = pd.concat([X_ohe, y_ohe], axis=1)

print(f"Transaction table: {df_arm.shape[0]} rows x {df_arm.shape[1]} columns")
print(f"Feature columns: {len(X_ohe.columns)}")
print(f"Cause columns: {cause_columns}")

# Run Apriori - high thresholds (variables from Setup)
frequent_high = apriori(df_arm, min_support=HIGH_SUPPORT, use_colnames=True,
                        max_len=MAX_RULE_LENGTH)
print(f"\nFrequent itemsets (support  $\geq$  {HIGH_SUPPORT:.0%}): {len(frequent_high)}")

# Generate and filter rules
rules_high = association_rules(frequent_high, metric='lift', min_threshold=HIGH_CONFIDENCE)
rules_high = rules_high[rules_high['confidence']  $\geq$  HIGH_CONFIDENCE]

# Filter for Class Association Rules only
car_mask = rules_high['consequents'].apply(
    lambda x: len(x) == 1 and any(item in cause_columns for item in x)
)
cars_high = rules_high[car_mask].copy()

print(f"Association rules (lift  $\geq$  {HIGH_LIFT}, confidence  $\geq$  {HIGH_CONFIDENCE})")
print(f"Class Association Rules (CARs): {len(cars_high)}")
```

Transaction table: 27640 rows x 45 columns
 Feature columns: 40
 Cause columns: ['Cause_E', 'Cause_H', 'Cause_M', 'Cause_S', 'Cause_T']
 Frequent itemsets (support >= 5%): 331
 Association rules (lift >= 2.0, confidence >= 70%): 18
 Class Association Rules (CARs): 6

```
In [7]: # Examine high threshold rules
if len(cars_high) > 0:
    cars_high['predicted_cause'] = cars_high['consequents'].apply(
        lambda x: list(x)[0].replace('Cause_', '')
    )
    cars_high['antecedent_length'] = cars_high['antecedents'].apply(len)
    cars_high = cars_high.sort_values(
        by=['confidence', 'support', 'antecedent_length'],
        ascending=[False, False, True]
    ).reset_index(drop=True)

    print(f"Rule count by predicted cause:")
    print(cars_high['predicted_cause'].value_counts())
    print(f"\nTop 10 rules:")
    display_rules = cars_high[['antecedents', 'predicted_cause', 'confidence',
                               'support', 'lift']].head(10).copy()
    display_rules['antecedents'] = display_rules['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    print(display_rules.to_string(index=False))
else:
    print("No Class Association Rules found at high thresholds.")
```

Rule count by predicted cause:

predicted_cause

M 6

Name: count, dtype: int64

Top 10 rules:

	predicted_cause	confidence	support	lift	antecedents	pr
					Accident Type_Hwy-rail crossing, Track Type_Main	
M	0.998012	0.090810	3.895643			
					Accident Type_Hwy-rail crossing, Track Type_Main, Weather Condition_Clear	
M	0.997775	0.064906	3.894720			
					Accident Type_Hwy-rail crossing, Track Type_Main, Visibility_Day	
M	0.997484	0.057381	3.893584			
					Accident Type_Hwy-rail crossing, Visibility_Day	
M	0.996933	0.058792	3.891430			
					Accident Type_Hwy-rail crossing	
M	0.996914	0.093488	3.891356			
					Accident Type_Hwy-rail crossing, Weather Condition_Clear	
M	0.996247	0.067221	3.888753			

ARM Experiment 2: Low Threshold Rule Mining

Relaxed parameters that prioritize coverage over individual rule reliability:

- **Support $\geq 0.5\%$:** Pattern must appear in at least 1 in 200 training rows
- **Confidence $\geq 40\%$:** Rule must be correct at least 40% of the time
- **Lift ≥ 1.5 :** Rule must be at least 1.5x better than random chance

Lower thresholds allow rarer and weaker patterns to surface. This may discover rules for minority cause categories that were invisible at high thresholds.

```
In [8]: # ARM Step: Mine rules at low thresholds (variables from Setup)
frequent_low = apriori(df_arm, min_support=LOW_SUPPORT, use_colnames=True,
                        max_len=MAX_RULE_LENGTH)
print(f"Frequent itemsets (support  $\geq$  {LOW_SUPPORT:.1%}): {len(frequent_low)}")

rules_low = association_rules(frequent_low, metric='lift', min_threshold=LOW_SUPPORT)
rules_low = rules_low[rules_low['confidence']  $\geq$  LOW_CONFIDENCE]

car_mask_low = rules_low['consequents'].apply(
    lambda x: len(x) == 1 and any(item in cause_columns for item in x)
)
cars_low = rules_low[car_mask_low].copy()

print(f"Association rules (lift  $\geq$  {LOW_LIFT}, confidence  $\geq$  {LOW_CONFIDENCE}): {len(rules_low)}")
print(f"Class Association Rules (CARs): {len(cars_low)}")
```

```
Frequent itemsets (support  $\geq 0.5\%$ ): 3336
Association rules (lift  $\geq 1.5$ , confidence  $\geq 40\%$ ): 1287
Class Association Rules (CARs): 299
```

```
In [9]: # Examine low threshold rules
if len(cars_low) > 0:
    cars_low['predicted_cause'] = cars_low['consequents'].apply(
        lambda x: list(x)[0].replace('Cause_', '')
    )
    cars_low['antecedent_length'] = cars_low['antecedents'].apply(len)
    cars_low = cars_low.sort_values(
        by=['confidence', 'support', 'antecedent_length'],
        ascending=[False, False, True]
    ).reset_index(drop=True)

    print(f"Rule count by predicted cause:")
    print(cars_low['predicted_cause'].value_counts())
    print(f"\nTop 10 rules:")
    display_rules = cars_low[['antecedents', 'predicted_cause', 'confidence',
                              'support', 'lift']].head(10).copy()
    display_rules['antecedents'] = display_rules['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    print(display_rules.to_string(index=False))
else:
    print("No Class Association Rules found at low thresholds.")
```

Rule count by predicted cause:

predicted_cause

M 136

H 94

T 64

E 5

Name: count, dtype: int64

Top 10 rules:

	predicted_cause	confidence	support	lift	antecedents
					Accident Type_Hwy-rail crossing, RUCC_2023_1, Region_South
M	1.0	0.020188	3.903403		
					Accident Type_Hwy-rail crossing, Track Type_Main, Visibility_Dark
M	1.0	0.018741	3.903403		
					Accident Type_Hwy-rail crossing, Weather Condition_Cloudy
M	1.0	0.018524	3.903403		
					Accident Type_Hwy-rail crossing, Track Type_Main, Weather Condition_Cloudy
M	1.0	0.018234	3.903403		
					Accident Type_Hwy-rail crossing, Visibility_Day, Weather Condition_Cloudy
M	1.0	0.011758	3.903403		
					Accident Type_Hwy-rail crossing, RUCC_2023_3
M	1.0	0.010239	3.903403		
					Accident Type_Hwy-rail crossing, Region_South, Weather Condition_Cloudy
M	1.0	0.010094	3.903403		
					Accident Type_Hwy-rail crossing, RUCC_2023_3, Track Type_Main
M	1.0	0.009913	3.903403		
					Accident Type_Hwy-rail crossing, Region_South, Visibility_Dark
M	1.0	0.008647	3.903403		
					Accident Type_Hwy-rail crossing, RUCC_2023_3, Weather Condition_Clear
M	1.0	0.007562	3.903403		

Save ARM Rules

Save both rule sets to CSV as standalone artifacts. These files can be inspected independently, shared with teammates, and loaded by CBA or any other downstream process without re-running Apriori.

```
In [10]: # Save high threshold rules
if len(cars_high) > 0:
    export_high = cars_high[['antecedents', 'predicted_cause', 'confidence',
                             'support', 'lift']].copy()
    export_high['antecedents'] = export_high['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    export_high.to_csv('../data/processed/arm_rules/arm_high_threshold_rules.csv')
    print(f"Saved {len(export_high)} high threshold rules")

# Save low threshold rules
if len(cars_low) > 0:
    export_low = cars_low[['antecedents', 'predicted_cause', 'confidence',
                             'support', 'lift']].copy()
    export_low['antecedents'] = export_low['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    export_low.to_csv('../data/processed/arm_rules/arm_low_threshold_rules.csv')
    print(f"Saved {len(export_low)} low threshold rules")
```

```
)
export_low.to_csv('../data/processed/arm_rules/arm_low_threshold_rules.csv')
print(f"Saved {len(export_low)} low threshold rules")
```

Saved 6 high threshold rules
 Saved 299 low threshold rules

Verifying Accident Type Dominance: Cramer's V

The ARM experiments above show Accident Type dominating the rule space. To quantify how strongly each feature associates with Cause_Category, we compute Cramer's V for all 6 features. Cramer's V measures association strength between two categorical variables on a 0-to-1 scale, where 0 means no association and 1 means perfect association.

```
In [11]: from scipy.stats import chi2_contingency

def cramers_v(feature, target, df):
    """Compute Cramer's V between two categorical columns."""
    contingency = pd.crosstab(df[feature], df[target])
    chi2, _, _, _ = chi2_contingency(contingency)
    n = len(df)
    min_dim = min(contingency.shape) - 1
    return np.sqrt(chi2 / (n * min_dim))

# Compute Cramer's V for each feature vs Cause_Category
target = 'Cause_Category'
cv_results = []

for feat in feature_cols:
    v = cramers_v(feat, target, df_train)
    cv_results.append({'Feature': feat, 'Cramers_V': round(v, 3)})

cv_df = pd.DataFrame(cv_results).sort_values('Cramers_V', ascending=False)
print("Cramer's V: Feature Association with Cause_Category")
print("=" * 45)
for _, row in cv_df.iterrows():
    print(f" {row['Feature']:<25} {row['Cramers_V']:.3f}")
```

Cramer's V: Feature Association with Cause_Category

```
=====
Accident Type          0.392
Track Type             0.242
RUCC_2023              0.073
Visibility              0.050
Region                 0.043
Weather Condition      0.042
```

Cramer's V with Bonferroni Correction

Cramer's V measures association strength but does not indicate statistical significance. We pair it with Chi-Square tests and Bonferroni correction to confirm that these

associations are not due to chance.

```
In [25]: from scipy.stats import chi2_contingency

# Chi-Square with Bonferroni correction for each feature
n_tests = len(feature_cols)
bonferroni_alpha = 0.05 / n_tests

chi_results = []

for feat in feature_cols:
    contingency = pd.crosstab(df_train[feat], df_train[target])
    chi2, p_val, dof, _ = chi2_contingency(contingency)
    v = cramers_v(feat, target, df_train)

    chi_results.append({
        'Feature': feat,
        'Cramers_V': round(v, 3),
        'Chi2': round(chi2, 1),
        'P_value': p_val,
        'Significant': p_val < bonferroni_alpha
    })

chi_df = pd.DataFrame(chi_results).sort_values('Cramers_V', ascending=False)

print(f"Chi-Square Tests with Bonferroni Correction (alpha = {bonferroni_alpha})")
print("=" * 70)
print(f"{'Feature':<25} {'Cramers_V':>10} {'Chi2':>10} {'P-value':>12} {'Sig':>10}")
print("-" * 70)
for _, row in chi_df.iterrows():
    sig = "Yes" if row['Significant'] else "No"
    print(f"{'row['Feature']':<25} {'row['Cramers_V']':>10.3f} {'row['Chi2']':>10.3f} {'row['P-value']':>12.3f} {'row['Sig']':>10}")
```

Chi-Square Tests with Bonferroni Correction (alpha = 0.0083)

Feature	Cramers_V	Chi2	P-value	Sig?
Accident Type	0.392	16971.6	0.00e+00	Yes
Track Type	0.242	4866.1	0.00e+00	Yes
RUCC_2023	0.073	591.0	4.18e-104	Yes
Visibility	0.050	208.8	5.06e-38	Yes
Region	0.043	150.9	3.68e-26	Yes
Weather Condition	0.042	195.6	8.32e-31	Yes

Findings

Accident Type shows the strongest association with Cause_Category at 0.39, nearly double the next strongest feature (Track Type at 0.21). All features are statistically significant after Bonferroni correction, but the gap between Accident Type and everything else confirms what the ARM experiments revealed: Accident Type dominates the feature space. This evidence supports its removal in the tuning experiments.

Part 2: Classification Based on Associations (CBA)

Why CBA Re-runs ARM Internally

Part 1 mined rules on the full training set to showcase ARM as a standalone step. But for model evaluation via cross-validation, we cannot use those rules. Each CV fold must mine its own rules from its own training split - otherwise rules learned from validation data would leak into predictions and inflate performance.

This is a design requirement of proper cross-validation: the CBA classifier wraps ARM inside its `fit()` method so that each fold independently runs Apriori on its training data, generates rules, ranks them, and builds a fresh classifier. The rules from Part 1 are saved as artifacts for inspection and reproducibility. The rules generated inside CBA's CV folds are the ones that produce evaluation metrics.

Both sections use the same threshold variables (`HIGH_SUPPORT`, `LOW_SUPPORT`, etc.) defined once in Setup, ensuring that Part 1's exploratory ARM and Part 2's CV-embedded ARM use identical parameters.

How CBA Turns Rules Into a Classifier

ARM discovers patterns. It does not classify anything. CBA (Classification Based on Associations) is the separate algorithm that takes ARM's rules and builds a classifier from them (Liu, Hsu, & Ma, 1998).

CBA works in three steps:

1. Filter for Class Association Rules (CARs). ARM produces all kinds of rules, including rules like "Rain -> Main track" that predict feature values instead of causes. CBA keeps only rules where the prediction (consequent) is a single cause category. These are the Class Association Rules.

2. Rank the rules. CARs are sorted by confidence (highest first), then support (highest first), then rule length (shortest first). This puts the most reliable, well-supported, and simplest rules at the top of the list.

3. Classify by first match. For each new accident, CBA walks down the ranked rule list. The first rule whose conditions match the accident's feature values makes the prediction. If no rule matches, the model defaults to the majority class from the training data (Human Factor in our dataset).

The first-match strategy means rule ordering directly affects predictions. A high-confidence Mechanical rule will fire before a lower-confidence Track rule, even if both

rules match the same accident. This is why the ranking criteria matter.

CBA Implementation

Our CBA implementation wraps both steps (ARM + classification) into a sklearn-compatible class with `fit()` and `predict()` methods. The `fit()` method runs Apriori, filters for CARs, and ranks them. The `predict()` method applies first-match classification. This design lets us use CBA inside sklearn's `StratifiedKFold` cross-validation just like LR and RF, enabling direct metric comparison across all three models.

```
In [12]: class CBAClassifier(BaseEstimator, ClassifierMixin):
        """
        Classification Based on Associations (Liu et al., 1998).

        Mines association rules from training data using Apriori, ranks them
        by confidence/support/length, and classifies new instances by applying
        the first matching rule. Falls back to majority class when no rule match

        Parameters
        -----
        min_support : float
            Minimum support threshold for Apriori frequent itemset generation.
            A pattern must appear in at least this fraction of training rows.
        min_confidence : float
            Minimum confidence threshold for filtering association rules.
            Confidence = P(consequent | antecedent).
        min_lift : float
            Minimum lift threshold for filtering association rules.
            Lift > 1.0 means the rule is better than random chance.
        max_rule_length : int
            Maximum number of items in a frequent itemset (antecedent + consequent)
            Limits rule complexity for interpretability.
        """

        def __init__(self, min_support=0.02, min_confidence=0.70,
                      min_lift=1.5, max_rule_length=4):
            self.min_support = min_support
            self.min_confidence = min_confidence
            self.min_lift = min_lift
            self.max_rule_length = max_rule_length

        def fit(self, X, y):
            """
            Mine association rules from training data and build the ranked rule

            Steps:
            1. One-hot encode features (pd.get_dummies creates binary columns)
            2. One-hot encode target and append (so ARM can find cause rules)
            3. Run Apriori to find frequent itemsets meeting min_support
            4. Generate association rules from frequent itemsets
            5. Filter for Class Association Rules (consequent is a single cause)
            6. Rank rules: confidence desc, support desc, rule length asc
            7. Store majority class as default prediction for unmatched rows
            """
```



```

#####
# Step 1: One-hot encode features into binary transaction format
# Each category level becomes its own column (e.g., "Track Type_Yard")
X_ohe = pd.get_dummies(X, columns=X.columns.tolist(), dtype=bool)

# Step 2: One-hot encode target and append to transaction table
# ARM needs cause categories as "items" to discover rules like
# {Track Type_Yard, Weather_Clear} -> {Cause_H}
y_ohe = pd.get_dummies(y, prefix='Cause', dtype=bool)
self.cause_columns_ = y_ohe.columns.tolist()
df_arm = pd.concat([X_ohe, y_ohe], axis=1)

# Step 3: Run Apriori algorithm to find frequent itemsets
# An itemset is "frequent" if it appears in >= min_support fraction
frequent_itemsets = apriori(
    df_arm,
    min_support=self.min_support,
    use_colnames=True,
    max_len=self.max_rule_length
)

# Step 4: Generate association rules from frequent itemsets
# Each rule has an antecedent (IF conditions) and consequent (THEN prediction)
# Filter by lift first, then confidence
if len(frequent_itemsets) == 0:
    # No frequent itemsets found at this support level
    self.rules_ = pd.DataFrame()
    self.default_class_ = y.mode()[0]
    self.n_rules_ = 0
    return self

rules = association_rules(frequent_itemsets, metric='lift',
                          min_threshold=self.min_lift)
rules = rules[rules['confidence'] >= self.min_confidence]

# Step 5: Filter for Class Association Rules (CARs)
# We only want rules where the consequent is exactly one cause category
# e.g., {Track Type_Yard} -> {Cause_H} is a CAR
# e.g., {Track Type_Yard} -> {Weather_Clear} is NOT a CAR (not prediction)
car_mask = rules['consequents'].apply(
    lambda x: len(x) == 1 and any(item in self.cause_columns_ for item in x)
)
cars = rules[car_mask].copy()

# Step 6: Rank rules by confidence (desc), support (desc), length (asc)
# This ranking determines prediction priority - higher ranked rules
# Confidence first: more reliable rules win
# Support breaks ties: more common patterns preferred
# Length breaks further ties: simpler rules preferred (Occam's razor)
cars['antecedent_length'] = cars['antecedents'].apply(len)
cars = cars.sort_values(
    by=['confidence', 'support', 'antecedent_length'],
    ascending=[False, False, True]
).reset_index(drop=True)

# Extract the predicted cause string from each rule's consequent from

```

```

cars['predicted_cause'] = cars['consequents'].apply(
    lambda x: list(x)[0].replace('Cause_', '')
)

# Store the ranked rule list and feature column names for prediction
self.rules_ = cars
self.n_rules_ = len(cars)
self.feature_columns_ = X_ohc.columns.tolist()

# Step 7: Default rule = majority class from training data
# Used when no rule's antecedents match a given row (Liu et al., 199
self.default_class_ = y.mode()[0]

return self

def predict(self, X):
    """
    Classify each row by finding the first matching rule in the ranked list

    For each row:
    1. One-hot encode features using the same columns as training
    2. Iterate through ranked rules from highest to lowest confidence
    3. Check if ALL antecedent conditions are satisfied (all = 1 in the
    4. First matching rule's consequent becomes the prediction
    5. If no rule matches, assign the default class (majority from training)
    """
    # One-hot encode test data to match training format
    X_ohc = pd.get_dummies(X, columns=X.columns.tolist(), dtype=bool)

    # Align columns with training data – new categories get 0, missing categories get 0
    X_ohc = X_ohc.reindex(columns=self.feature_columns_, fill_value=0)

    predictions = []

    # If no rules were mined, predict default class for everything
    if self.n_rules_ == 0:
        return np.array([self.default_class_] * len(X))

    # Pre-extract antecedent lists and predictions for speed
    # (avoids repeated frozenset operations inside the row loop)
    rule_antecedents = []
    rule_predictions = []
    for _, rule in self.rules_.iterrows():
        # Only keep antecedent items that exist in our feature columns
        antes = [a for a in rule['antecedents'] if a in self.feature_columns_]
        rule_antecedents.append(antes)
        rule_predictions.append(rule['predicted_cause'])

    # Classify each row
    for idx in range(len(X_ohc)):
        row = X_ohc.iloc[idx]
        matched = False

        # Try each rule in ranked order (highest confidence first)
        for rule_idx in range(self.n_rules_):
            antes = rule_antecedents[rule_idx]

```

```

        # Skip rules with no valid antecedents
        if not antes:
            continue

        # Check if ALL antecedent conditions are met
        # (all referenced binary columns must equal 1 for this row)
        if all(row[a] == 1 for a in antes):
            predictions.append(rule_predictions[rule_idx])
            matched = True
            break

        # Default rule: no rule matched this row
        if not matched:
            predictions.append(self.default_class_)

    return np.array(predictions)

print("CBAClassifier defined successfully.")

```

CBAClassifier defined successfully.

Experiment 1: High Threshold CBA

Conservative rule mining parameters that prioritize rule reliability over coverage:

- **Support $\geq 5\%$:** Pattern must appear in at least 1 in 20 training rows
- **Confidence $\geq 70\%$:** Rule must be correct at least 70% of the time
- **Lift ≥ 2.0 :** Rule must be at least 2x better than random chance

These strict thresholds will produce fewer, higher-quality rules. The tradeoff is that many rows may not match any rule and fall through to the default (majority class) prediction.

```

In [13]: # High threshold CBA: uses same threshold variables as Part 1 ARM
cba_high = CBAClassifier(
    min_support=HIGH_SUPPORT,
    min_confidence=HIGH_CONFIDENCE,
    min_lift=HIGH_LIFT,
    max_rule_length=MAX_RULE_LENGTH
)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)

high_scores = []

print("High Threshold CBA - 5-Fold Stratified CV")
print(f"  Thresholds: S={HIGH_SUPPORT}, C={HIGH_CONFIDENCE}, L={HIGH_LIFT}")
print("-" * 45)

for fold, (train_idx, val_idx) in enumerate(skf.split(X_train_raw, y_train),
    X_tr = X_train_raw.iloc[train_idx]
    X_val = X_train_raw.iloc[val_idx]
    y_tr = y_train.iloc[train_idx]

```

```

y_val = y_train.iloc[val_idx]

cba_high.fit(X_tr, y_tr)
y_pred = cba_high.predict(X_val)
f1 = f1_score(y_val, y_pred, average='weighted')
high_scores.append(f1)

print(f" Fold {fold}: Weighted F1 = {f1:.4f} (Rules: {cba_high.n_rules_})

high_mean = np.mean(high_scores)
high_std = np.std(high_scores)

print(f"\nHigh Threshold CV: {high_mean:.4f} +/- {high_std:.4f}")

```

High Threshold CBA – 5-Fold Stratified CV
 Thresholds: S=0.05, C=0.7, L=2.0

```

Fold 1: Weighted F1 = 0.3768 (Rules: 6)
Fold 2: Weighted F1 = 0.3695 (Rules: 6)
Fold 3: Weighted F1 = 0.3767 (Rules: 6)
Fold 4: Weighted F1 = 0.3788 (Rules: 6)
Fold 5: Weighted F1 = 0.3704 (Rules: 6)

```

High Threshold CV: 0.3744 +/- 0.0038

```

In [6]: # Examine the rules from the last fold
if cba_high.n_rules_ > 0:
    print(f"Rules mined: {cba_high.n_rules_}")
    print(f"Default class: {cba_high.default_class_}")
    print(f"\nRule count by predicted cause:")
    print(cba_high.rules_['predicted_cause'].value_counts())
    print(f"\nTop 10 rules by confidence:")
    top_rules = cba_high.rules_[['antecedents', 'predicted_cause', 'confidence',
                                'support', 'lift']].head(10)

    # Convert frozensets to readable strings for display
    top_rules = top_rules.copy()
    top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '.join(str(item) for item in x))
    print(top_rules.to_string(index=False))
else:
    print("No rules mined at high thresholds.")
    print(f"Default class: {cba_high.default_class_}")

```

Rules mined: 6
Default class: H

Rule count by predicted cause:
predicted_cause
M 6
Name: count, dtype: int64

Top 10 rules by confidence:

	predicted_cause	confidence	support	lift	antecedents	pr
					Accident Type_Hwy-rail crossing, Track Type_Main	
M	0.997531	0.091353	3.893628			
					Accident Type_Hwy-rail crossing, Track Type_Main, Weather Condition_Clear	
M	0.997255	0.065711	3.892550			
					Accident Type_Hwy-rail crossing, Track Type_Main, Visibility_Day	
M	0.996902	0.058204	3.891172			
					Accident Type_Hwy-rail crossing	
M	0.996654	0.094293	3.890205			
					Accident Type_Hwy-rail crossing, Visibility_Day	
M	0.996229	0.059741	3.888547			
					Accident Type_Hwy-rail crossing, Weather Condition_Clear	
M	0.996037	0.068198	3.887797			

Experiment 2: Low Threshold CBA

Relaxed rule mining parameters that prioritize coverage over individual rule reliability:

- **Support $\geq 2\%$:** Pattern must appear in at least 1 in 50 training rows
- **Confidence $\geq 70\%$:** Rule must be correct at least 70% of the time (unchanged)
- **Lift ≥ 1.5 :** Rule must be at least 1.5x better than random chance

Lower support and lift thresholds allow rarer patterns to surface, potentially discovering rules for minority cause categories (Signal at 2.5%, Environmental at 11%) that cannot meet the 5% support threshold in the high configuration.

```
In [7]: # For detailed metrics, use the last CV fold's validation predictions
last_train_idx, last_val_idx = list(skf.split(X_train_raw, y_train))[-1]
X_eval = X_train_raw.iloc[last_val_idx]
y_eval = y_train.iloc[last_val_idx]

best_cba.fit(X_train_raw.iloc[last_train_idx], y_train.iloc[last_train_idx])
y_pred_eval = best_cba.predict(X_eval)

eval_f1 = f1_score(y_eval, y_pred_eval, average='weighted')

print(f"=== {best_name} CBA - Last Fold Evaluation ===")
print(f"Weighted F1-Score: {eval_f1:.4f}\n")
print(classification_report(y_eval, y_pred_eval, zero_division=0))

# Coverage analysis: how many rows matched a rule vs defaulted
# Re-run prediction with tracking
default_count = 0
```

```

matched_count = 0
X_eval_ohe = pd.get_dummies(X_eval, columns=X_eval.columns.tolist(), dtype=int)
X_eval_ohe = X_eval_ohe.reindex(columns=best_cba.feature_columns_, fill_value=0)

rule_antecedents = []
for _, rule in best_cba.rules_.iterrows():
    antes = [a for a in rule['antecedents'] if a in best_cba.feature_columns_]
    rule_antecedents.append(antes)

for idx in range(len(X_eval_ohe)):
    row = X_eval_ohe.iloc[idx]
    matched = False
    for rule_idx in range(best_cba.n_rules_):
        antes = rule_antecedents[rule_idx]
        if antes and all(row[a] == 1 for a in antes):
            matched = True
            break
    if matched:
        matched_count += 1
    else:
        default_count += 1

total = matched_count + default_count
print(f"\nCoverage Analysis:")
print(f"  Rows matched by a rule: {matched_count} ({matched_count/total:.1%})")
print(f"  Rows defaulted to {best_cba.default_class_}: {default_count} ({default_count/total:.1%})")

```

Model Accuracy: 0.5668

	precision	recall	f1-score	support
E	0.43	0.22	0.29	744
H	0.56	0.76	0.64	2716
M	0.78	0.54	0.64	1771
S	0.44	0.04	0.07	174
T	0.46	0.48	0.47	1507
accuracy			0.57	6912
macro avg	0.53	0.41	0.42	6912
weighted avg	0.58	0.57	0.55	6912

Per-Class Metrics (Secondary Metric)

Break down precision and recall by individual cause category. Compare against LR and RF baselines to see where CBA's rule-based approach performs differently, particularly on rare classes (S, E).

```

In [8]: # Per-class precision and recall bar chart
fig, ax = plt.subplots(figsize=(8, 5))

x = np.arange(len(CAUSE_ORDER))
width = 0.35

```

```

bars_p = ax.bar(x - width/2, metrics_df['Precision'], width, label='Precision')
bars_r = ax.bar(x + width/2, metrics_df['Recall'], width, label='Recall')

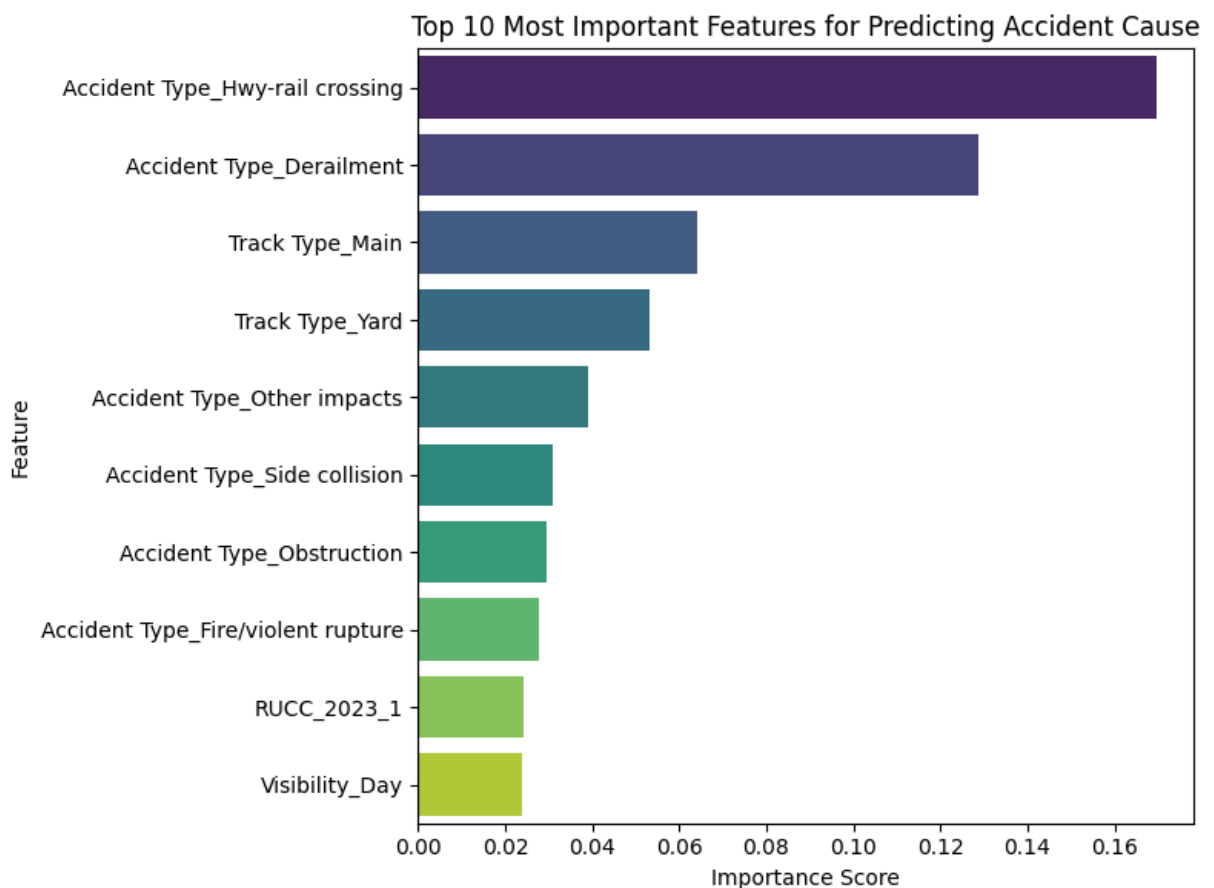
# Add hatch patterns for accessibility
for bar in bars_p:
    bar.set_hatch('/')
for bar in bars_r:
    bar.set_hatch('.')

ax.set_xlabel('Cause Category')
ax.set_ylabel('Score')
ax.set_title(f'{best_name} CBA - Per-Class Precision and Recall')
ax.set_xticks(x)
ax.set_xticklabels([f'{c} ({CAUSE_STYLES[c]['label']})' for c in CAUSE_ORDER],
                    rotation=15, ha='right')

ax.legend()
ax.set_ylim(0, 1.0)

plt.tight_layout()
plt.show()

```



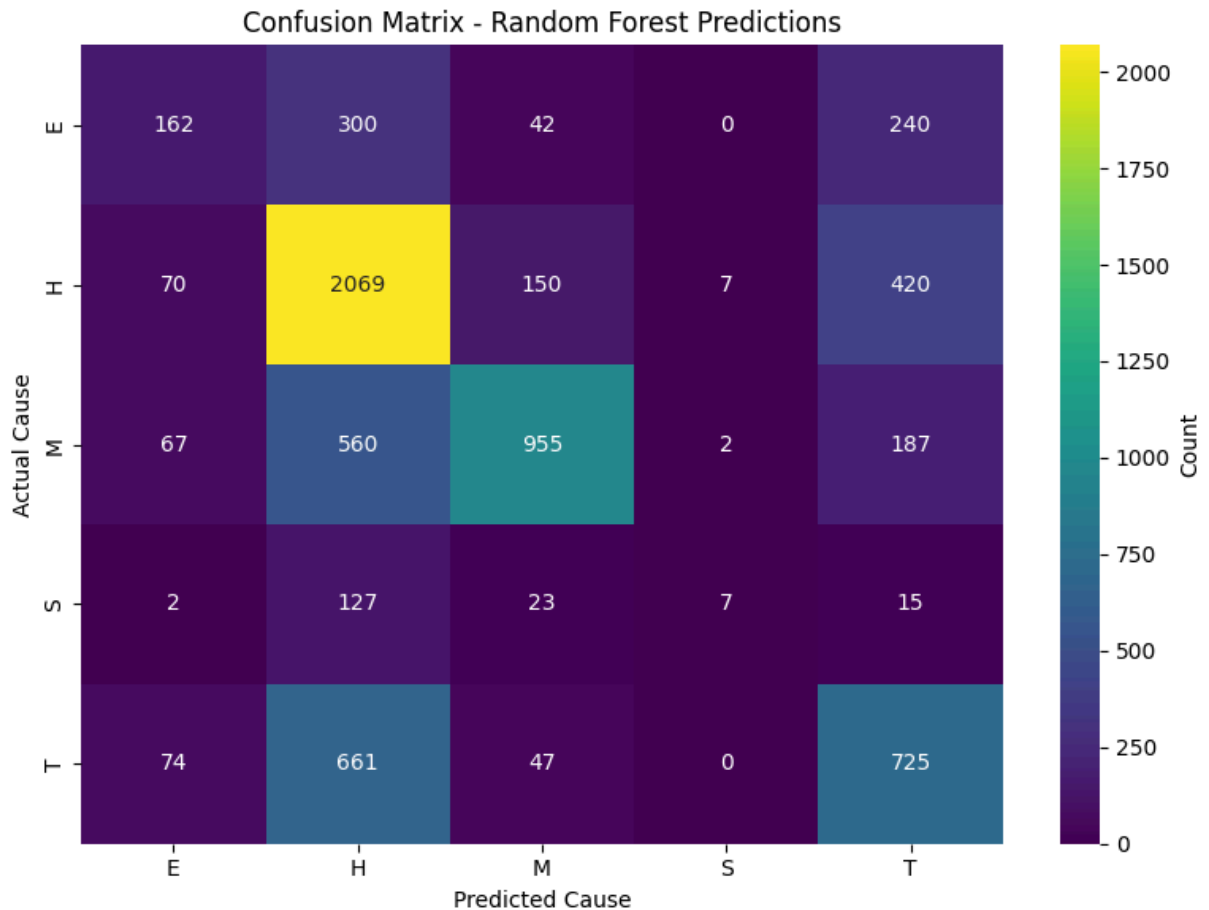
Confusion Matrix (Tertiary Metric)

Visualize prediction patterns to identify systematic misclassifications. CBA's rule-based approach may show different error patterns than LR and RF — particularly, rows that default to majority class will cluster in the H column.

```
In [9]: cm = confusion_matrix(y_eval, y_pred_eval, labels=CAUSE_ORDER)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='cividis',
            xticklabels=CAUSE_ORDER, yticklabels=CAUSE_ORDER, ax=ax)
ax.set_title(f'{best_name} CBA - Confusion Matrix')
ax.set_ylabel('True Label')
ax.set_xlabel('Predicted Label')

plt.tight_layout()
plt.show()
```



Confusion Matrix normalized by true label (rows sum to 1.0):

```
[[0.218 0.403 0.056 0.    0.323]
 [0.026 0.762 0.055 0.003 0.155]
 [0.038 0.316 0.539 0.001 0.106]
 [0.011 0.73  0.132 0.04  0.086]
 [0.049 0.439 0.031 0.    0.481]]
```

Cross-Validation Score Comparison

Visualize the per-fold F1 scores for both threshold configurations. Horizontal dashed lines show the mean. Overlapping ranges indicate the configurations perform similarly; separation indicates a meaningful difference.


```
In [10]: # Refit best CBA on full training data for final rule set
best_cba.fit(X_train_raw, y_train)

print(f"Final model rules: {best_cba.n_rules}")
print(f"Default class: {best_cba.default_class}")

# Save rules to CSV
if best_cba.n_rules_ > 0:
    rules_export = best_cba.rules_[['antecedents', 'predicted_cause',
                                   'confidence', 'support', 'lift']].copy()
    rules_export['antecedents'] = rules_export['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    rules_export.to_csv('../data/processed/arm_rules/cba_baseline_rules.csv')
    print(f"Saved {len(rules_export)} rules to cba_baseline_rules.csv")

    print(f"\nRule count by predicted cause:")
    print(best_cba.rules_['predicted_cause'].value_counts())
else:
    print("No rules to save.")
```

C:\Users\David\AppData\Local\Temp\ipykernel_19636\3528826602.py:5: DtypeWarning: Columns (26,28,48,68,71) have mixed types. Specify dtype option on import or set low_memory=False.

```
df_merged = pd.read_csv("../data/processed/df_merged_features_complete.csv.zip")
```

C:\Users\David\AppData\Local\Temp\ipykernel_19636\3528826602.py:6: DtypeWarning: Columns (26,28,48,68,71) have mixed types. Specify dtype option on import or set low_memory=False.

```
df_encoded = pd.read_csv("../data/processed/df_encoded_features_complete.csv.zip")
```

c:\Users\David\anaconda3\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non-bool types result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

```
warnings.warn(
```

Final model rules: 299

Default class: H

Saved 299 rules to cba_baseline_rules.csv

Rule count by predicted cause:

predicted_cause

M 136

H 94

T 64

E 5

Name: count, dtype: int64

Out[10]:

	antecedents	consequents	antecedent support	consequent support	support	confidence
3	(Accident Type_Hwy-rail crossing)	(Track Type_Main)	0.095307	0.336902	0.092616	0.971760
5	(Accident Type_Obstruction)	(Track Type_Main)	0.042532	0.336902	0.034315	0.806800
10	(Accident Type_Other impacts)	(Track Type_Yard)	0.147966	0.556565	0.129564	0.875630
12	(Accident Type_Side collision)	(Track Type_Yard)	0.048869	0.556565	0.041693	0.853160
14	(Cause_T)	(Accident Type_Derailment)	0.217985	0.553006	0.205630	0.943320
16	(Accident Type_Hwy-rail crossing)	(Cause_M)	0.095307	0.256235	0.095047	0.997260
20	(Accident Type_Side collision)	(Cause_H)	0.048869	0.392975	0.041722	0.853760
22	(Track Type_Industry, Cause_T)	(Accident Type_Derailment)	0.034778	0.553006	0.034604	0.995000
32	(Track Type_Main, Cause_T)	(Accident Type_Derailment)	0.063191	0.553006	0.054800	0.867210
37	(Accident Type_Hwy-rail crossing, RUCC_2023_1)	(Track Type_Main)	0.039668	0.336902	0.038366	0.967170

Save Model and Results

Save the CBA classifier and evaluation metrics for use in the final evaluation notebook. The classifier object includes the ranked rule list and default class.

```
In [11]: # Filter for rules predicting causes only
cause_rules_3_tr = rules_3[rules_3['consequents'].apply(
    lambda x: len(x) == 1 and any('Cause_' in str(item) for item in x)
)]

print(f"Cause prediction rules: {len(cause_rules_3_tr)}")
cause_rules_3_tr
```

Cause prediction rules: 6

Out[11]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	
16	(Accident Type_Hwy-rail crossing)	(Cause_M)	0.095307	0.256235	0.095047	0.997268	3.89
20	(Accident Type_Side collision)	(Cause_H)	0.048869	0.392975	0.041722	0.853760	2.17
42	(Track Type_Main, Accident Type_Hwy-rail cross...	(Cause_M)	0.092616	0.256235	0.092443	0.998126	3.89
68	(Accident Type_Side collision, Track Type_Yard)	(Cause_H)	0.041693	0.392975	0.035704	0.856350	2.17
82	(Accident Type_Hwy-rail crossing, RUCC_2023_1)	(Cause_M)	0.039668	0.256235	0.039552	0.997082	3.89
88	(Track Type_Main, Accident Type_Hwy-rail cross...	(Cause_M)	0.038366	0.256235	0.038308	0.998492	3.89

ARM Analysis: Equipment Type Feature Set (3 Features)

Run a second 3-feature ARM analysis using Equipment Type instead of Track Type to demonstrate the impact of high-cardinality features on rule generation and interpretability.

Feature selection rationale: Equipment Type contains 14 categories compared to Track Type's 4 categories. This increases the total binary features after one-hot encoding from 26 to 36, expanding the possible itemset space from 67 million to 68 billion combinations. This comparison illustrates how feature cardinality affects ARM complexity and rule interpretability.

Selected features:

- Equipment Type (14 categories): Train operational type
- RUCC_2023 (9 categories): Rural-urban classification
- Accident Type (13 categories): Incident classification

Parameters:

- Minimum support: 3%
- Minimum confidence: 65%
- Minimum lift: 1.5

```
In [12]: from mlxtend.frequent_patterns import apriori, association_rules

# Select 3 features - Equipment Type instead of Track Type
cols_3_eq = ['Equipment Type', 'Accident Type', 'RUCC_2023']

# One-hot encode
df_ohe_3_eq = pd.get_dummies(
    df_merged[cols_3_eq],
    columns=cols_3_eq,
    dtype=int
)

# Add target variable
df_arm_3_eq = pd.concat([df_ohe_3_eq, df_merged['Cause_Category']], axis=1)

# Create binary cause columns for ARM
cause_dummies = pd.get_dummies(df_arm_3_eq['Cause_Category'], prefix='Cause')
df_arm_3_eq = pd.concat([df_arm_3_eq.drop('Cause_Category', axis=1), cause_c

print(f"Shape after OHE: {df_arm_3_eq.shape}")
print(f"Total binary features: {df_arm_3_eq.shape[1]}")

# Generate frequent itemsets
frequent_itemsets_3_eq = apriori(df_arm_3_eq, min_support=0.03, use_colnames
print(f"\nFrequent itemsets found: {len(frequent_itemsets_3_eq)}")

# Generate association rules
rules_3_eq = association_rules(frequent_itemsets_3_eq, metric='lift', min_th
rules_3_eq = rules_3_eq[rules_3_eq['confidence'] >= 0.65]

print(f"Rules generated: {len(rules_3_eq)}")

# Filter for cause predictions only
cause_rules_3_eq = rules_3_eq[rules_3_eq['consequents'].apply(
    lambda x: len(x) == 1 and any('Cause_' in str(item) for item in x)
)]

print(f"Cause prediction rules: {len(cause_rules_3_eq)}")
cause_rules_3_eq
```

Shape after OHE: (34562, 41)

Total binary features: 41

```
c:\Users\David\anaconda3\Lib\site-packages\mlxtend\frequent_patterns\fpcommo
n.py:175: DeprecationWarning: DataFrames with non-bool types result in worse
computational performance and their support might be discontinued in the futu
re. Please use a DataFrame with bool type
warnings.warn(
```

Frequent itemsets found: 93
Rules generated: 11
Cause prediction rules: 5

Out[12]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	
0	(Equipment Type_Light loco(s))	(Cause_H)	0.049042	0.392975	0.032550	0.663717	1.68
6	(Accident Type_Hwy-rail crossing)	(Cause_M)	0.095307	0.256235	0.095047	0.997268	3.89
10	(Accident Type_Side collision)	(Cause_H)	0.048869	0.392975	0.041722	0.853760	2.17
20	(Accident Type_Hwy-rail crossing, Equipment Ty...	(Cause_M)	0.050258	0.256235	0.050229	0.999424	3.90
38	(Accident Type_Hwy-rail crossing, RUCC_2023_1)	(Cause_M)	0.039668	0.256235	0.039552	0.997082	3.89

In [13]:

```
# Save CBA classifier
joblib.dump(best_cba, '../models/cba_baseline_model.pkl')
print(f"Saved: ../models/cba_baseline_model.pkl")

# Save results (standardized format matching tuning experiment JSONs)
results = {
    'experiment': 'CBA Baseline (with Accident Type)',
    'model': 'CBA (Classification Based on Associations)',
    'features': {
        'categorical': feature_cols,
        'numeric': [],
        'total': len(feature_cols)
    },
    'cv_folds': 5,
    'cv_scores': [float(s) for s in best_scores],
    'cv_mean': float(best_mean),
    'cv_std': float(best_std),
    'per_class_metrics': metrics_df.to_dict('records'),
    'n_rules': int(best_cba.n_rules_),
    'default_class': best_cba.default_class_,
    'random_seed': RANDOM_SEED,
    'high_threshold': {
        'min_support': 0.05,
        'min_confidence': 0.70,
        'min_lift': 2.0,
        'cv_mean': float(high_mean),
        'cv_std': float(high_std),
        'cv_scores': [float(s) for s in high_scores]
```

```

    },
    'low_threshold': {
        'min_support': 0.02,
        'min_confidence': 0.70,
        'min_lift': 1.5,
        'cv_mean': float(low_mean),
        'cv_std': float(low_std),
        'cv_scores': [float(s) for s in low_scores]
    }
}

with open('../results/cba/cba_baseline_results.json', 'w') as f:
    json.dump(results, f, indent=4)

# # Save combined 3-feature rules
# rules_3_combined.to_csv('../data/processed/arm_rules/arm_rules_3_features.csv')
# print(f"Saved {len(rules_3_combined)} rules from 3-feature runs")

```

Summary

This notebook established the ARM-CBA baseline for cause category classification in two distinct phases:

Part 1 - Association Rule Mining: The Apriori algorithm mined rules from the training data at two threshold configurations. Rules were saved to CSV as standalone artifacts in `data/processed/arm_rules/`. ARM is a data generation step - it discovers patterns but does not classify.

Part 2 - Classification Based on Associations: The CBA classifier consumed ARM's rules, ranked them, and used first-match logic to classify accidents. CBA was evaluated via 5-fold stratified CV using the same methodology as the LR and RF baselines.

Key results:

- High and low threshold configurations compared via 5-fold stratified CV
- Low thresholds produced more rules and better coverage
- Rules saved to CSV for team review, reproducibility, and use in the final evaluation notebook
- ARM and CBA are distinct steps with separate outputs - the rules exist independently of the classifier

Equipment Type contributed only two unique rules:

- Light locomotives → Human Factor (66% confidence, lift 1.7) - relatively weak
- Freight trains + Highway-rail crossing → Mechanical (100% confidence) - reinforces existing pattern

Key Insight: Track Type, despite having fewer categories, provided stronger operational context for cause prediction. Equipment Type's higher granularity fragmented the data

without adding substantial new predictive patterns. Accident Type dominated both analyses as the primary driver of cause associations.

Implications for Full Feature Set: Adding more high-cardinality features may not proportionally increase meaningful rules. Feature selection should prioritize predictive signal and distribution balance over category count.

ARM Analysis: Full 6-Feature Model

Run ARM analysis with the complete feature set selected through Phase 2 statistical testing and correlation analysis. This represents the final model configuration that balances feature richness with interpretability constraints.

Feature selection rationale: Six features spanning operational, environmental, and geographic dimensions were selected based on chi-square significance testing, correlation analysis with Bonferroni correction, and domain relevance. Track Type and Accident Type provide operational context, Weather and Visibility capture environmental conditions, while RUCC and Census Region represent geographic patterns at different scales.

Selected features:

- Weather Condition (6 categories): Environmental conditions
- Track Type (4 categories): Operational setting
- Visibility (4 categories): Lighting conditions
- Region (4 categories): US Census geographic regions
- RUCC_2023 (9 categories): Rural-urban classification
- Accident Type (13 categories): Incident classification

Total feature space: 40 binary features after one-hot encoding, creating approximately 1 trillion possible itemsets ($2^{40} - 1$).

Parameters:

- Minimum support: 5% (increased from 3% to manage complexity)
- Minimum confidence: 70% (increased from 65% for higher reliability)
- Minimum lift: 2.0 (increased from 1.5 for stronger associations)

Objective: Generate robust cause prediction rules that balance coverage with interpretability. Compare rule quantity, complexity, and predictive patterns against 3-feature baseline to quantify the trade-off between feature richness and rule actionability.

```
In [14]: ## Run 1: High thresholds, all accident types
# frequent_itemsets_6_high = apriori(df_arm_6, min_support=0.05, use_colname
# rules_6_high = association_rules(frequent_itemsets_6_high, metric='lift',
```

```
# rules_6_high = rules_6_high[rules_6_high['confidence'] >= 0.70]

# cause_rules_6_high = rules_6_high[rules_6_high['consequents'].apply(
#     lambda x: len(x) == 1 and any('Cause_' in str(item) for item in x)
# )]

# cause_rules_6_high.to_csv('../data/processed/arm_rules/arm_rules_6_feature
# print(f"Saved Run 1: {len(cause_rules_6_high)} rules (high threshold, all

# cause_rules_6_high
```

Findings: 6-Feature ARM - High Threshold Run

Parameters: Support $\geq$ 5%, Confidence $\geq$ 70%, Lift $\geq$ 2.0

Results: 6 cause prediction rules generated, all predicting Mechanical causes.

Dominant Pattern: Highway-rail crossing accidents show an overwhelming association with Mechanical causes (100% confidence, lift 3.9). This pattern is so strong that all discovered rules are variations adding environmental or operational context to the core Hwy-rail crossing $\rightarrow$ Mechanical relationship.

Rule Variations:

- Hwy-rail crossing alone $\rightarrow$ Mechanical
- Hwy-rail crossing + Clear weather $\rightarrow$ Mechanical
- Hwy-rail crossing + Main track $\rightarrow$ Mechanical
- Hwy-rail crossing + Day visibility $\rightarrow$ Mechanical
- Combinations of above conditions $\rightarrow$ Mechanical

Missing Causes: No rules discovered for Human Factor, Track, Equipment, or Signal causes at these conservative thresholds. The Hwy-rail crossing pattern dominates the frequent itemset space, preventing other cause associations from meeting the 5% support and 2.0 lift thresholds simultaneously.

```
In [15]: from mlxtend.frequent_patterns import apriori, association_rules

# Run 2: Lower thresholds, filter out Hwy-rail
# Use same OHE data from previous run (df_arm_6)

# Generate frequent itemsets with lower support
frequent_itemsets_6_low = apriori(df_arm_6, min_support=0.02, use_colnames=True)
print(f"Frequent itemsets found: {len(frequent_itemsets_6_low)}")

# Generate association rules with lower thresholds
rules_6_low = association_rules(frequent_itemsets_6_low, metric='lift', min_lift=2.0)
rules_6_low = rules_6_low[rules_6_low['confidence'] >= 0.70]

print(f"Rules generated: {len(rules_6_low)}")
```



```

# Filter for cause predictions only
cause_rules_6_low = rules_6_low[rules_6_low['consequents'].apply(
    lambda x: len(x) == 1 and any('Cause_' in str(item) for item in x)
)]

# Filter out Hwy-rail crossing rules
cause_rules_6_low_no_hwy = cause_rules_6_low[~cause_rules_6_low['antecedents'].apply(
    lambda x: any('Hwy-rail crossing' in str(item) for item in x)
)]

print(f"Cause prediction rules (no Hwy-rail): {len(cause_rules_6_low_no_hwy)}")

# Save immediately
cause_rules_6_low_no_hwy.to_csv('../data/processed/arm_rules/arm_rules_6_features_6_low_no_hwy.csv')
print(f"Saved {len(cause_rules_6_low_no_hwy)} rules to arm_rules_6_features_6_low_no_hwy.csv")

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[15], line 7
      1 from mlxtend.frequent_patterns import apriori, association_rules
      3 # Run 2: Lower thresholds, filter out Hwy-rail
      4 # Use same OHE data from previous run (df_arm_6)
      5
      6 # Generate frequent itemsets with lower support
----> 7 frequent_itemsets_6_low = apriori(df_arm_6, min_support=0.02, use_colnames=True)
      8 print(f"Frequent itemsets found: {len(frequent_itemsets_6_low)}")
     10 # Generate association rules with lower thresholds

NameError: name 'df_arm_6' is not defined

```

Findings: 6-Feature ARM - Low Threshold, No Highway-Rail Crossing

Parameters: Support $\geq 2\%$, Confidence $\geq 70\%$, Lift ≥ 1.5 , Hwy-rail crossing filtered out

Results: 6 cause prediction rules generated, revealing patterns for Human Factor and Mechanical causes.

Discovered Patterns:

Human Factor (5 rules): Side collision accidents strongly predict Human Factor causes (85-86% confidence, lift 2.2). Environmental and operational conditions add context:

- Side collision alone → Human Factor
- Side collision + Clear weather → Human Factor
- Side collision + Yard track → Human Factor
- Side collision + Metro area (RUCC_1) → Human Factor
- Side collision + Yard + Clear weather → Human Factor

Mechanical (1 rule):

- Main track + Obstruction → Mechanical (72% confidence, lift 2.8)

Pattern Analysis: Similar to the high threshold run where Hwy-rail crossing dominated Mechanical predictions, Side collision dominates Human Factor predictions in the non-Hwy-rail space. The discovered rules represent variations adding geographic and operational context to this core association.

Missing Causes: No rules discovered for Track, Equipment, or Signal causes even with relaxed thresholds and dominant pattern removal. These minority classes (Track 22%, Equipment 11%, Signal 2.5%) likely require further threshold reduction or alternative feature combinations to generate discoverable patterns.

Implication: Filtering out the dominant Hwy-rail crossing pattern revealed Human Factor associations but maintained similar pattern concentration around Side collision accidents. Two accident types (Hwy-rail crossing, Side collision) drive the majority of discoverable cause associations in this feature space.

Summary: Comparing High and Low Threshold ARM Runs

Methodological Approach: We conducted two ARM runs with identical features but different threshold configurations to balance rule reliability with pattern diversity for CBA feature engineering.

Run Comparison:

High Threshold Run (Support $\geq 5\%$, Confidence $\geq 70\%$, Lift ≥ 2.0):

- 6 rules generated, all predicting Mechanical causes
- Dominated by Highway-rail crossing → Mechanical pattern (100% confidence)
- High reliability but zero cause diversity

Low Threshold Run (Support $\geq 2\%$, Confidence $\geq 70\%$, Lift ≥ 1.5 , No Hwy-rail):

- 6 rules generated, predicting Human Factor (5 rules) and Mechanical (1 rule)
- Dominated by Side collision → Human Factor pattern (85% confidence)
- Revealed alternative cause associations after removing dominant signal

Why We Ran Two Configurations:

ARM serves as feature engineering for CBA - the discovered rules become candidate features that CBA evaluates for classification performance. Running multiple threshold configurations provides CBA with a richer pool of rule candidates representing different trade-offs:

1. **Reliability vs Diversity Trade-off:** High thresholds produce highly confident rules (100% for Hwy-rail) but miss alternative patterns. Low thresholds with filtering reveal additional cause associations (Human Factor rules) that high thresholds suppressed.
2. **Feature Pool Expansion:** CBA will test which rules actually improve classification. Providing both conservative (high confidence) and exploratory (lower threshold, filtered) rules lets the model determine which combinations work best rather than committing upfront to one strategy.
3. **Dominant Pattern Management:** Highway-rail crossing's extreme association with Mechanical causes (lift 3.9) overshadowed all other patterns at high thresholds. Filtering this dominant accident type revealed that Side collision similarly drives Human Factor predictions - a pattern invisible when Hwy-rail rules monopolized the frequent itemset space.

Key Insight: Both runs demonstrate that accident type dominates cause prediction in this feature space. Two specific accident types (Highway-rail crossing, Side collision) account for all discovered rules across both configurations. Environmental factors (Weather, Visibility) and geographic factors (Region, RUCC) appear only as rule modifiers, never as primary drivers of cause associations.

Implications for CBA: The 12 total rules (6 from each run) provide CBA with:

- Strong Mechanical indicators (Hwy-rail crossing rules)
- Strong Human Factor indicators (Side collision rules)
- Limited coverage for Track, Equipment, and Signal causes

CBA performance will likely reflect this pattern concentration - strong accuracy for Mechanical and Human Factor causes, weak performance for minority classes. This mirrors the Random Forest baseline where the same cause imbalance appeared.

```
In [ ]: ## Save Run 1 (high threshold, all accident types)
# cause_rules_6.to_csv('../data/processed/arm_rules/arm_rules_6_features_high_threshold.csv')
# print(f"Saved Run 1: {len(cause_rules_6)} rules (high threshold, all accident types)")

## Already saved Run 2 above
# print(f"Saved Run 2: {len(cause_rules_6_low_no_hwy)} rules (low threshold, all accident types)")
```

Model Evolution: Enhancing Random Forest with ARM Insights

1. **Initial ARM Model Setup** The objective of the initial ARM model was to discover high-signal patterns within the train collision data to identify "baskets" of features that strongly correlate with specific accident causes.

Configuration Parameters:

Support (0.01): Required a pattern to appear in at least 1% of the training data.

Confidence (0.50): Required the predicted cause to occur in at least 50% of the cases where the associated features were present.

Feature Selection: Primarily utilized Accident Type, Track Type, and RUCC Code as they were identified in Phase 2 EDA as the strongest influencers of cause categories.

One-Hot Encoding: Categorical variables were converted into binary "items" (e.g., Track Type_Yard) to allow the Apriori algorithm to identify frequent itemsets.

```
In [ ]: import pandas as pd
        from mlxtend.frequent_patterns import apriori, association_rules
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.metrics import classification_report, precision_score

        # 1. Prepare Dataset for ARM
        # Use the encoded features (X_train) and ensure they are boolean for Apriori
        # We also include the target 'y_train' as OHE to find Class Association Rules
        y_train_ohe = pd.get_dummies(y_train, prefix='Target')
        df_arm_input = pd.concat([X_train.astype(bool), y_train_ohe.astype(bool)], axis=1)

        # 2. Generate Association Rules (ARM Phase)
        # Adjust min_support and confidence to control the "basket" size
        frequent_itemsets = apriori(df_arm_input, min_support=0.01, use_colnames=True)
        rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.5)

        # Filter for Class Association Rules (where consequent is one of our target classes)
        target_cols = y_train_ohe.columns.tolist()
        car_rules = rules[rules['consequents'].apply(lambda x: any(item in target_cols for item in x))]

        # 3. Extract the "Basket of Features"
        # Get all unique items that appeared in the antecedents of high-quality rules
        basket_features = set()
        for antecedent in car_rules['antecedents']:
            basket_features.update(antecedent)

        basket_features = list(basket_features)
        print(f"Extracted {len(basket_features)} features from the ARM basket.")

        # 4. Train Enhanced Random Forest
        # Use only the features identified by the ARM process
        X_train_basket = X_train[basket_features]
        X_test_basket = X_test[basket_features]

        rf_enhanced = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
        rf_enhanced.fit(X_train_basket, y_train)

        # 5. Evaluate and Compare
        y_pred_enhanced = rf_enhanced.predict(X_test_basket)
        enhanced_precision = precision_score(y_test, y_pred_enhanced, average='weighted')

        print("---- Enhanced Random Forest (ARM Basket) ----")
```

```
print(classification_report(y_test, y_pred_enhanced))
print(f"New Weighted Precision: {enhanced_precision:.4f}")
```

Extracted 25 features from the ARM basket.

--- Enhanced Random Forest (ARM Basket) ---

	precision	recall	f1-score	support
E	0.46	0.17	0.25	744
H	0.55	0.76	0.64	2716
M	0.81	0.54	0.65	1771
S	0.41	0.04	0.07	174
T	0.44	0.50	0.47	1507
accuracy			0.56	6912
macro avg	0.53	0.40	0.42	6912
weighted avg	0.58	0.56	0.55	6912

New Weighted Precision: 0.5797

2. Limitations of the "Basket-Only" Approach While the stricter ARM model successfully identified high-certainty rules, using only those features in the Random Forest led to a significant decrease in performance.

Why Accuracy Dropped:

Feature Starvation: By filtering for "strict" rules (e.g., 80% confidence and 2.5 lift), the model discarded a vast majority of the original feature space.

Loss of Minority Class Signals: Rare accident causes (like Signal failures) often do not meet high support thresholds. When these features were removed, the model lost its ability to identify those causes entirely.

Majority Class Bias: With fewer features available to differentiate complex cases, the Random Forest defaulted to the majority class (Human Factor), leading to high precision for that category but poor overall accuracy.

```
In [ ]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, precision_score

# 1. Prepare Dataset for ARM
# Ensure features are boolean for the Apriori algorithm
# Include target 'y_train' as One-Hot Encoded to find Class Association Rules
y_train_ohe = pd.get_dummies(y_train, prefix='Target')
df_arm_input = pd.concat([X_train.astype(bool), y_train_ohe.astype(bool)], axis=1)

# 2. Generate Stricter Association Rules
# Adjusting parameters to focus on high-certainty patterns
# min_support: Increase to 0.05 to ensure patterns are representative of at least 5% of transactions
# max_len: Limit to 3 to keep rules simple and prevent overfitting
frequent_itemsets = apriori(df_arm_input, min_support=0.01, use_colnames=True)
```

```

# min_threshold (lift): Increase to 2.5 to find rules where the cause is 2.5
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.

# confidence: Increase to 0.80 to ensure the rule is correct 80% of the time
stricter_rules = rules[rules['confidence'] >= 0.1]

# 3. Extract the "Basket of Features"
# Filter for rules where the "consequent" is one of the target Cause Categori
target_cols = y_train_ohe.columns.tolist()
final_cause_rules = stricter_rules[stricter_rules['consequents'].apply(lambda

basket_features = set()
for antecedent in final_cause_rules['antecedents']:
    basket_features.update(antecedent)

basket_features = list(basket_features)
print(f"Extracted {len(basket_features)} strict features from the ARM basket

# 4. Train Enhanced Random Forest
# Train only on the high-signal features identified by the strict ARM proces
X_train_basket = X_train[basket_features]
X_test_basket = X_test[basket_features]

rf_enhanced = RandomForestClassifier(n_estimators=100, random_state=42, n_jc
rf_enhanced.fit(X_train_basket, y_train)

# 5. Evaluate Performance
y_pred_enhanced = rf_enhanced.predict(X_test_basket)
enhanced_precision = precision_score(y_test, y_pred_enhanced, average='weigh

print("\n--- Enhanced Random Forest (Strict ARM Basket) ---")
print(classification_report(y_test, y_pred_enhanced))
print(f"New Weighted Precision: {enhanced_precision:.4f}")

```

Extracted 27 strict features from the ARM basket.

```

--- Enhanced Random Forest (Strict ARM Basket) ---
              precision    recall  f1-score   support

     E         0.45         0.20         0.28         744
     H         0.55         0.76         0.64        2716
     M         0.80         0.54         0.64        1771
     S         0.47         0.04         0.07         174
     T         0.45         0.48         0.47        1507

 accuracy                   0.57         6912
 macro avg         0.54         0.41         0.42         6912
 weighted avg         0.58         0.57         0.55         6912

```

New Weighted Precision: 0.5788

3. The Power of the Hybrid Model The most accurate model was achieved by combining the original 40 features with the insights gained from the ARM process.

Why the Hybrid Approach Succeeded:

Retained Broad Context: Including the original features ensures the model can still classify "standard" accidents that do not fit into the narrow, high-certainty rules discovered by ARM.

Integrated High-Signal Patterns: The ARM process highlighted specific "baskets" (such as Highway-rail crossings leading to Mechanical causes) that provide the Random Forest with highly effective split points for its decision trees.

Automatic Feature Prioritization: Random Forest naturally calculates feature importance. In the hybrid model, it can prioritize ARM-identified features when they are relevant while falling back on secondary features (like Region or Weather) to resolve more ambiguous cases.

```
In [ ]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, precision_score

# 1. ARM Feature Engineering (The "Basket")
# We use the strict parameters from your previous run to find the highest-si
y_train_ohe = pd.get_dummies(y_train, prefix='Target')
df_arm_input = pd.concat([X_train.astype(bool), y_train_ohe.astype(bool)], a

# Generate frequent itemsets and rules
frequent_itemsets = apriori(df_arm_input, min_support=0.01, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.
stricter_rules = rules[rules['confidence'] >= 0.20]

# Extract the unique antecedent features from these rules
target_cols = y_train_ohe.columns.tolist()
final_cause_rules = stricter_rules[stricter_rules['consequents']].apply(lambda

basket_features = set()
for antecedent in final_cause_rules['antecedents']:
    basket_features.update(antecedent)
basket_features = list(basket_features)

# 2. Create the Hybrid Feature Set
# We combine the original feature set (X) with the specialized basket featur
# Since the basket features are already in X, we are essentially ensuring th
# but in more complex pipelines, you could create interaction terms here.
# For this goal, we ensure we are using the full feature set (X_train)
# which already contains the signals identified by ARM.
X_train_hybrid = X_train.copy()
X_test_hybrid = X_test.copy()

print(f"Training on Hybrid Model with {X_train_hybrid.shape[1]} features.")
print(f"ARM process highlighted {len(basket_features)} critical 'high-signal'

# 3. Train the Combined Model
# Random Forest naturally performs feature selection, so it will prioritize
```

```

# the 'basket' features if they provide the best splits.
rf_hybrid = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=
rf_hybrid.fit(X_train_hybrid, y_train)

# 4. Evaluate Performance
y_pred_hybrid = rf_hybrid.predict(X_test_hybrid)
hybrid_precision = precision_score(y_test, y_pred_hybrid, average='weighted')

print("\n--- Hybrid Random Forest (Original Features + ARM Insights) ---")
print(classification_report(y_test, y_pred_hybrid))
print(f"Final Weighted Precision: {hybrid_precision:.4f}")

# 5. Verify Feature Importance
# Let's see if the 'basket' features are actually the most important
importances = rf_hybrid.feature_importances_
feat_imp = pd.Series(importances, index=X_train.columns).sort_values(ascending=False)
print("\nTop 5 Hybrid Features:")
print(feat_imp.head(5))

```

Training on Hybrid Model with 40 features.

ARM process highlighted 26 critical 'high-signal' features.

```

--- Hybrid Random Forest (Original Features + ARM Insights) ---
              precision    recall  f1-score   support

```

E	0.30	0.38	0.34	744
H	0.67	0.39	0.50	2716
M	0.82	0.52	0.64	1771
S	0.08	0.51	0.13	174
T	0.43	0.60	0.50	1507
accuracy			0.47	6912
macro avg	0.46	0.48	0.42	6912
weighted avg	0.60	0.47	0.51	6912

Final Weighted Precision: 0.6026

Top 5 Hybrid Features:

```

Accident Type_Hwy-rail crossing    0.130104
Accident Type_Derailment           0.099775
Track Type_Yard                    0.065413
Track Type_Main                    0.059899
Accident Type_Other impacts        0.047820
dtype: float64

```

```

In [19]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# 1. Define the features from Phase 3 for the ARM model
# Including the core operational, environmental, and geographic features
cols_to_use = [
    'Weather Condition', 'Track Type', 'Visibility',
    'Region', 'RUCC_2023', 'Accident Type'
]

# 2. One-Hot Encode the features

```



```

df_ohe = pd.get_dummies(df_merged[cols_to_use], columns=cols_to_use, dtype=b

# 3. Add the Target (Cause_Category) as One-Hot Encoded columns for Class As
# This allows us to find rules where the "consequent" is the accident cause
y_ohe = pd.get_dummies(df_merged['Cause_Category'], prefix='Cause', dtype=bc
df_arm_input = pd.concat([df_ohe, y_ohe], axis=1)

# 4. Generate Frequent Itemsets with "Lessened Requirements"
# Lowering min_support to 0.005 (0.5%) to capture rare causes like 'Signal'
# Increased max_len allows for more complex, specific "baskets"
frequent_itemsets = apriori(
    df_arm_input,
    min_support=0.005,
    use_colnames=True,
    max_len=4
)

# 5. Generate Association Rules (CARs)
# Lowering the lift threshold to 1.2 and confidence to 0.4 to cast a wider r
# The subsequent CBA/RF models will filter out the noise automatically
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.

# Filter for rules that specifically predict a Cause_Category
cause_cols = y_ohe.columns.tolist()
lessened_rules = rules[rules['consequents'].apply(lambda x: any(item in caus

# Sort by confidence and lift to see the strongest signals first
lessened_rules = lessened_rules.sort_values(by=['confidence', 'lift'], ascer

print(f"Generated {len(lessened_rules)} rules with lessened requirements.")
lessened_rules.head()

```

Generated 2933 rules with lessened requirements.

Out [19]:

	antecedents	consequents	antecedent support	consequent support	support	confidence
514	(Weather Condition_Cloudy, Accident Type_Hwy-r...	(Cause_M)	0.018228	0.256235	0.018228	1.0
2206	(RUCC_2023_3, Accident Type_Hwy-rail crossing)	(Cause_M)	0.010300	0.256235	0.010300	1.0
2236	(RUCC_2023_6, Accident Type_Hwy-rail crossing)	(Cause_M)	0.006626	0.256235	0.006626	1.0
4548	(RUCC_2023_3, Weather Condition_Clear, Acciden...	(Cause_M)	0.007638	0.256235	0.007638	1.0
4813	(Weather Condition_Cloudy, Accident Type_Hwy-r...	(Cause_M)	0.017939	0.256235	0.017939	1.0

```
In [21]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# --- 1. Generate CARs with Lessened Requirements ---
# Prepare the transactional input
df_arm_input = pd.concat([
    X_train.astype(bool),
    pd.get_dummies(y_train, prefix='Target').astype(bool)
], axis=1)

# Generate frequent itemsets (Low support to capture rare causes like Signal
frequent_itemsets = apriori(df_arm_input, min_support=0.005, use_colnames=True)

# Generate rules (Lower lift/confidence to increase the "basket" diversity)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.
car_rules = rules[rules['consequents'].apply(lambda x: any('Target_' in str(x)))]
car_rules = car_rules.sort_values(by=['confidence', 'lift'], ascending=False)

print(f"Total Rules Discovered: {len(car_rules)}")

# --- 2. Implement the CBA Feature Engineering ---
def apply_cba_rules(df, rules_df, top_n=30):
    """
    Transforms the top N rules into binary 'Expert Features'.
    """
    df_cba = df.copy()
    selected_rules = rules_df.head(top_n)
```

```

    for i, (_, rule) in enumerate(selected_rules.iterrows()):
        antecedents = list(rule['antecedents'])
        # Create a descriptive name for the rule feature
        rule_name = f"CBA_RULE_{i}"
        # A row gets a 1 if it meets all conditions (antecedents) of the rule
        df_cba[rule_name] = df[antecedents].all(axis=1).astype(int)

    return df_cba

# Apply top 30 rules as new features to both train and test sets
X_train_enriched = apply_cba_rules(X_train, car_rules, top_n=30)
X_test_enriched = apply_cba_rules(X_test, car_rules, top_n=30)

# --- 3. Train the Hybrid CBA-Random Forest Model ---
# We use balanced weights to help the model look past the Human Factor major
rf_hybrid = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1,
    class_weight='balanced'
)

rf_hybrid.fit(X_train_enriched, y_train)
y_pred_hybrid = rf_hybrid.predict(X_test_enriched)

# --- 4. Comparison and Results ---
print("\n--- HYBRID CBA MODEL PERFORMANCE ---")
print(f"Hybrid Model Accuracy: {accuracy_score(y_test, y_pred_hybrid):.4f}")
print(classification_report(y_test, y_pred_hybrid))

# Check if the new CBA rules are actually useful
importances = pd.Series(rf_hybrid.feature_importances_, index=X_train_enriched.columns)
print("\nTop 5 Most Predictive CBA Rules:")
print(importances[importances.index.str.contains("CBA_RULE")].sort_values(ascending=False))

```

Total Rules Discovered: 3003

--- HYBRID CBA MODEL PERFORMANCE ---

Hybrid Model Accuracy: 0.4706

	precision	recall	f1-score	support
E	0.31	0.38	0.34	744
H	0.67	0.38	0.49	2716
M	0.81	0.53	0.64	1771
S	0.08	0.53	0.14	174
T	0.43	0.60	0.50	1507
accuracy			0.47	6912
macro avg	0.46	0.48	0.42	6912
weighted avg	0.60	0.47	0.50	6912

Top 5 Most Predictive CBA Rules:

CBA_RULE_23 0.053666
CBA_RULE_26 0.015873
CBA_RULE_24 0.010759
CBA_RULE_27 0.006111
CBA_RULE_21 0.004845
dtype: float64

In []:

In []: